Ruta Fantasma

Memoria del proyecto Scrum — Desarrollo de Interfaces

Product Owner: Daniel Martín Scrum Master: Aitor
Desarrollo: Asier García Catalán, Daniel Rodríguez, Adam

1º DAM — Curso 2026-2027

Índice

1 Introducción **1**

2 El producto **1**
2.1 Visión 1
2.2 Usuarios 1
2.3 Product Backlog inicial 2

3 Equipo y forma de trabajo **2**
3.1 Roles 2
3.2 Tecnologías (stack) 2
3.3 Herramientas 2
3.4 Ceremonias 3

4 Sprint 1 **3**
4.1 Sprint Planning 3
4.2 Trabajo realizado 3
4.3 Configuración del repositorio y del flujo de trabajo 3
4.4 DT2 · Preparación y publicación: proyecto Angular con zonas y PWA . . 4
4.5 Organización del árbol del proyecto 5
4.6 Carpeta de diseño para los mockups 6
4.7 PS-26 · Pantalla con la lista de rutas 6
4.8 PS-20 · Pantallas de registro e inicio de sesión 8
4.9 PS-21 · Servicio HTTP base de Angular 10
4.10 Flujo de trabajo con ramas y Pull Requests 11

5 Conclusiones **12**

1 Introducción

Esta memoria recoge el desarrollo del proyecto **Ruta Fantasma**, realizado en la asignatura de Desarrollo de Interfaces siguiendo la metodología Scrum.

(Completad aquí: objetivo del trabajo, motivación de la idea y qué se espera conseguir al final.)

2 El producto

2.1 Visión

Ruta Fantasma es una aplicación de reloj para correr contra tu mejor marca. En pantalla aparecen dos puntos: tú y tu “sombra” (tu mejor carrera en ese recorrido). Durante la carrera, la app avisa por sectores en verde (vas mejor) o en rojo (vas peor), como los tiempos por sector de la Fórmula 1.

2.2 Usuarios

(Quién va a usar la app y qué necesita.)

2.3 Product Backlog inicial

ID	Historia de usuario	Prioridad	Estimación
HU-01	Como corredor, quiero ver mi posición y la de mi sombra para saber si voy mejor o peor	Alta	*(pts)*
HU-02	Como corredor, quiero recibir avisos verdes/rojos por sector para ajustar el ritmo	Alta	*(pts)*
HU-03	*(…)*		

3 Equipo y forma de trabajo

3.1 Roles

Rol	Persona	Responsabilidad
Product Owner	Daniel Martín	Define y prioriza el Product Backlog
Scrum Master	Aitor	Facilita las ceremonias y elimina impedimentos
Desarrollador	Asier García Catalán	Desarrollo
Desarrollador	Daniel Rodríguez	Desarrollo
Desarrollador	Adam	Desarrollo

3.2 Tecnologías (stack)

Parte	Tecnología	Motivo
Frontend	Angular + TypeScript	Vista en clase; permite simular la pantalla del reloj en el navegador
Backend	Node.js + Express	JavaScript en servidor, conocido por el equipo
Base de datos	MongoDB	Guarda de forma natural las rutas (listas de puntos GPS y tiempos)

3.3 Herramientas

- **GitHub:** repositorio común del código y de esta memoria.
- **GitHub Issues y Projects:** Product Backlog y tablero del sprint (Por hacer / En curso / Hecho).
- **Claude Code:** asistente de programación que usa cada desarrollador. Todos comparten el mismo contexto mediante el archivo `CLAUDE.md` del repositorio.

- **Generación automática de la memoria:** cada avance se documenta en Markdown y GitHub Actions genera este PDF en cada subida.

3.4 Ceremonias

- **Sprint Planning:** al inicio de cada sprint.
- **Daily:** *(frecuencia real del equipo)*.
- **Sprint Review y Retrospectiva:** al final de cada sprint; se resumen en el apartado de cada sprint.

4 Sprint 1

Fechas: *(inicio — fin)*

Objetivo del sprint: *(una frase)*

4.1 Sprint Planning

Historia / tarea	Responsable	Estimación
(HU-01 …)	*(nombre)*	*(pts)*

4.2 Trabajo realizado

4.3 Configuración del repositorio y del flujo de trabajo

Autor: Asier García · **Fecha:** 13/10/2026 · **Issue:** —

4.3.1 Qué se ha hecho

Se ha preparado el repositorio común para que los tres desarrolladores trabajen con el mismo contexto y para que la memoria se genere sola.

4.3.2 Cómo se ha hecho

- Se creó el archivo `CLAUDE.md` con el contexto del proyecto y las reglas del equipo. El asistente de cada desarrollador lo lee al empezar, de modo que los tres trabajan con la misma información.
- Se añadieron dos comandos, `/empezar` y `/terminar`, que automatizan actualizar el repositorio al empezar y documentar, hacer commit y subir el trabajo al terminar.
- La memoria se escribe en Markdown dentro de `docs/memoria/`. Cada tarea terminada genera un archivo propio, lo que evita conflictos entre compañeros.
- Un flujo de GitHub Actions convierte todos esos archivos en `docs/memoria.pdf` cada vez que se sube un cambio a la rama principal.

4.3.3 Por qué

Reduce el tiempo dedicado a tareas repetitivas (subir cambios, maquetar la memoria) y garantiza que la documentación esté siempre al día con el código.

4.3.4 Problemas encontrados

Ninguno por ahora.

4.4 DT2 · Preparación y publicación: proyecto Angular con zonas y PWA

Autor: Asier García · **Fecha:** 14/10/2026 · **Issue:** DT2

4.4.1 Qué se ha hecho

Se ha creado la base del frontend de Ruta Fantasma: un proyecto Angular dividido en tres zonas independientes (`/reloj`, `/app` y `/web`) y configurado como aplicación web progresiva (PWA), de modo que se puede instalar en el móvil y funciona sin conexión. Cada zona tiene ya una primera pantalla de ejemplo sobre la que el resto del equipo puede seguir trabajando.

4.4.2 Cómo se ha hecho

- **Creación del proyecto.** Con Angular CLI 22 se generó el proyecto en `src/frontend`, con enrutado, CSS y sin renderizado en servidor (`ng new ruta-fantasma --routing --style=css --ssr=false`).
- **Zonas con carga diferida.** En `src/app/app.routes.ts` se definieron las tres zonas mediante `loadChildren`. Cada una vive en su propia carpeta `src/app/zonas/<zona>/` con su archivo de rutas (`reloj.routes.ts`, `app.routes.ts`, `web.routes.ts`) y un componente de inicio. La raíz `/` y cualquier dirección desconocida redirigen a `/web`.
  - **`/web`:** página pública de presentación del producto, con titular y tres tarjetas de características.
  - **`/app`:** diseño de móvil con la lista “Mis rutas” (vacía de momento) y un menú inferior.
  - **`/reloj`:** esfera redonda que simula la pantalla del reloj, con los dos puntos (corredor y sombra), el tiempo y la diferencia del sector. El borde cambia a verde o rojo según se vaya mejor o peor. Usa *signals* de Angular y datos de ejemplo hasta que se conecte el GPS.
- **Estilos comunes.** En `src/styles.css` se definieron variables CSS (colores de fondo, morado “fantasma”, verde y rojo de sector) para que las tres zonas tengan una imagen coherente.
- **PWA.** Con `ng add @angular/pwa` se añadieron el *service worker* (`ngsw-config.json`), el manifiesto y los iconos. Después se personalizó el manifiesto (nombre “Ruta Fantasma”, idioma español, colores del tema) y el `index.html` (idioma, título, descripción, `theme-color` e icono para iOS).
- **Pruebas.**
  - **Pruebas unitarias:** se escribieron con Vitest para comprobar que cada ruta carga su zona, que las redirecciones funcionan y que el reloj formatea bien el tiempo (`12:34`) y la diferencia (`-2,4 s`). Pasan las 10.
  - **Pruebas en el navegador:** se comprobaron las tres zonas en tamaño de escritorio y de móvil. Además, sobre la versión de producción se verificó que el *service worker* se instala, guarda la aplicación en caché y que `/reloj` se abre con el servidor apagado.

4.4.3 Por qué

- **Tres zonas separadas:** cada desarrollador puede trabajar en una zona sin tocar los archivos de los demás, lo que reduce los conflictos en Git. Además, con la carga diferida el navegador solo descarga el código de la zona que se está usando. La compilación genera un fichero distinto por zona, de 1 a 3 kB cada uno.
- **PWA:** Ruta Fantasma se usa corriendo por la calle, donde la conexión puede fallar. Con el *service worker* la aplicación sigue funcionando sin red y puede instalarse como si fuera una app nativa, sin tener que publicarla en una tienda de aplicaciones.
- **Angular 22 con componentes *standalone* y *signals*:** es la forma recomendada actualmente por Angular y simplifica el código.

4.4.4 Problemas encontrados

- Los tests generados por defecto fallaban porque los componentes usan `routerLink` y el test no tenía router. Se resolvió añadiendo `provideRouter` en cada test.
- En las pruebas de rutas se comprobaba mal qué componente aparecía en pantalla. Se cambió a `RouterTestingHarness.navigateByUrl(url, Componente)`, que verifica directamente que se muestra el componente esperado.
- El *service worker* no se activa con `npm start` (modo desarrollo), solo en la versión de producción (`ng build`). Es el comportamiento normal de Angular y queda anotado en `CLAUDE.md`.

4.5 Organización del árbol del proyecto

Autor: Asier García · **Fecha:** 20/10/2026 · **Issue:** —

4.5.1 Qué se ha hecho

Se ha definido la estructura de carpetas completa del repositorio, tanto del frontend como del backend, para que cada parte del código tenga un lugar fijo antes de empezar a desarrollar las historias de usuario.

4.5.2 Cómo se ha hecho

- En el frontend (`src/frontend/src/app/`) se añadieron, junto a las `zonas/` ya existentes, dos carpetas nuevas: `core/` (servicios y modelos de datos que se usan en toda la aplicación) y `compartido/` (componentes y utilidades reutilizables entre zonas).
- Se creó el esqueleto del backend en `src/backend/`, organizado por capas: `api/` (endpoints), `controladores/`, `servicios/` (lógica de negocio), `modelos/` (MongoDB), además de `config/`, `middlewares/`, `utilidades/` y `tests/`.
- Se añadió un `.env.example` con las variables necesarias y un `README.md` del backend explicando cada carpeta.
- Se actualizó el árbol de `CLAUDE.md` para que los asistentes de todo el equipo conozcan la nueva estructura.

4.5.3 Por qué

Con una estructura acordada desde el principio, los desarrolladores saben dónde colocar cada archivo y se reducen los conflictos al trabajar en paralelo. La separación por capas del backend facilita además probar y modificar cada parte por separado.

4.5.4 Problemas encontrados

Ninguno.

4.6 Carpeta de diseño para los mockups

Autor: Asier García · **Fecha:** 21/10/2026 · **Issue:** —

4.6.1 Qué se ha hecho

Se ha creado en el repositorio la carpeta `docs/diseno/`, destinada a guardar los mockups de las pantallas de Ruta Fantasma antes de programarlas. Incluye una guía que explica cómo exportar y nombrar los diseños, y una tabla para seguir el estado de cada pantalla.

4.6.2 Cómo se ha hecho

- Se creó `docs/diseno/mockups/`, donde se subirán las imágenes PNG exportadas desde Figma.
- Se añadió `docs/diseno/README.md` con el espacio para el enlace al archivo de Figma, los pasos para exportar un mockup, la regla de nombres (`zona-pantalla.png`, con las mismas zonas que el frontend: `reloj`, `app` y `web`) y una tabla con las cinco pantallas previstas para el primer diseño y su estado (Pendiente, En revisión o Validado).
- Se actualizó el árbol del repositorio y la tabla de decisiones de `CLAUDE.md`.

4.6.3 Por qué

Figma guarda los diseños en la nube, por lo que no quedarían registrados en el repositorio. Subir las imágenes exportadas permite que todo el equipo y el profesor consulten los diseños directamente desde GitHub y que quede constancia de cómo evolucionan. Usar los mismos nombres de zona que el frontend facilita relacionar cada mockup con el código de su pantalla, y la columna de estado deja claro qué diseños ha validado ya el Product Owner.

4.6.4 Problemas encontrados

Se descartó el nombre `diseño` para la carpeta porque las tildes en los nombres de carpetas pueden dar problemas en algunos sistemas y herramientas; se usó `diseno`.

4.7 PS-26 · Pantalla con la lista de rutas

Autor: Asier García · **Fecha:** 21/10/2026 · **Issue:** PS-26

4.7.1 Qué se ha hecho

Se ha creado la pantalla “Mis rutas” (`/app`), que muestra las carreras guardadas en el backend. De cada ruta enseña el nombre, la distancia en kilómetros, el tiempo de la “sombra” (lo que se tardó en esa carrera) y la fecha. La pantalla contempla los cuatro estados posibles: cargando, error, lista vacía y lista con datos.

4.7.2 Cómo se ha hecho

- **Modelos** (`core/modelos/ruta.ts`). Las interfaces `Ruta` y `PuntoGps` copian exactamente los DTO del backend Spring Boot de Adam (`RutaResponse` y `PuntoGps`), para que los datos encajen sin conversiones.
- **Servicio `Rutas`** (`core/servicios/rutas.ts`). Usa el servicio `Api` de la tarea PS-21 para llamar a `GET /api/rutas` y `GET /api/rutas/{id}`.
- **Utilidades** (`compartido/utilidades/rutas.ts`). Son funciones puras, reutilizables por otras zonas (por ejemplo, el reloj):
  - `distanciaKm`: suma la distancia entre cada par de puntos GPS con la fórmula del semiverseno (*haversine*), que tiene en cuenta la curvatura de la Tierra.
  - `duracionSegundos`: calcula el tiempo entre el primer y el último punto.
  - Funciones de formato en español: `27:41`, `5,2 km` y `18 oct 2026`.
- **Componente `ListaRutas`** (`zonas/app/lista-rutas/`). Guarda el estado de la pantalla en *signals* y usa un `computed` que ordena las rutas de la más reciente a la más antigua y prepara los textos.
  - Mientras carga, muestra tres tarjetas “esqueleto” que laten. La animación se desactiva si el usuario tiene activada la opción de reducir el movimiento.
  - Si falla, muestra el mensaje del interceptor de errores y un botón **Reintentar**.
- **Diseño.** Las tarjetas siguen el mockup “Elegir ruta” de Daniel (`docs/diseno/mockups/`): nombre en negrita, “sombra mm:ss” debajo y los kilómetros a la derecha.
- **Estructura de la zona `/app`.** `AppInicio` pasa a ser el marco de la zona (cabecera con la sesión y menú inferior) y contiene un `router-outlet`. La lista es su ruta hija, así las próximas pantallas de la app reutilizarán ese marco sin repetirlo.
- **Pruebas.**
  - **Pruebas unitarias:** se añadieron 12 nuevas (utilidades, servicio y pantalla en sus cuatro estados, incluido el botón Reintentar). En total pasan 47.
  - **Prueba en el navegador:** se usó una API falsa con el mismo formato que el backend, a tamaño de móvil. Se comprobaron la lista con tres rutas, el error con reintento y la lista vacía, y que cada petición lleva el token `Authorization: Bearer` de la tarea PS-20.

4.7.3 Por qué

- **Calcular la distancia y el tiempo en el frontend:** el backend solo guarda los puntos GPS, así que no hace falta cambiar la API para mostrar estos datos.
- **Funciones de cálculo en `compartido/`:** el reloj también necesitará calcular distancias y tiempos durante la carrera, y así no se duplica el código.
- **Mostrar siempre un estado claro:** nunca aparece una pantalla en blanco. El usuario sabe en todo momento si está cargando, si no hay datos o si ha fallado algo, y puede reintentarlo.

4.7.4 Problemas encontrados

- Al principio un test no compilaba porque se pasaba un valor de tipo unknown a `flush()` de `HttpTestingController`. Se corrigió tipando el parámetro como `Ruta[] | null`.
- Como el backend no puede ejecutarse en este equipo (no tiene Java ni MongoDB), se usó una API falsa en Node.js que devuelve rutas con el mismo formato que `RutaResponse`.

4.8 PS-20 · Pantallas de registro e inicio de sesión

Autor: Asier García · **Fecha:** 21/10/2026 · **Issue:** PS-20

4.8.1 Qué se ha hecho

Se han creado las pantallas de inicio de sesión (`/app/login`) y de registro (`/app/registro`) en la zona de la app móvil, junto con el servicio que gestiona la sesión del usuario. La cabecera de la app muestra ahora "Hola, " y un botón **Salir** cuando hay sesión iniciada, o un enlace **Entrar** cuando no la hay.

4.8.2 Cómo se ha hecho

- **Formularios reactivos** (`ReactiveFormsModule`) con validaciones en el propio navegador:
  - **Login:** email obligatorio y con formato válido, y contraseña obligatoria.
  - **Registro:** nombre obligatorio (máximo 40 caracteres), email válido, contraseña de al menos 6 caracteres y confirmación. Un validador propio, `passwordsIguales`, comprueba que las dos contraseñas coinciden.
  - Los mensajes de error aparecen debajo de cada campo cuando el usuario ya lo ha tocado. Si se pulsa el botón con errores, se marcan todos los campos.
- **Servicio `Auth`** (`core/servicios/auth.ts`). Usa el servicio `Api` de la tarea PS-21 para llamar a `POST /api/auth/login` y `POST /api/auth/registro`. Guarda la sesión (token y usuario) en *signals* y en `localStorage`, para que siga abierta al recargar la página. Si `localStorage` está bloqueado o contiene datos corruptos, la aplicación sigue funcionando.
- **Interceptor `authInterceptor`.** Añade la cabecera `Authorization: Bearer <token>` a las peticiones que van a nuestra API, y solo a esas, para no enviar el token a otros servidores.
- **Guard `authGuard`** (`core/guardias/`). Queda preparado para proteger pantallas: si no hay sesión, redirige a `/app/login?volver=<página>` y, al entrar, el login devuelve al usuario a esa página.
- **Estados de la pantalla.** Mientras se espera al servidor, el botón se desactiva y cambia su texto ("Entrando…"). Los errores del servidor (credenciales incorrectas, email ya registrado, sin conexión) se muestran en un recuadro rojo con el mensaje en español que proporciona el interceptor de errores.
- **Estilos compartidos.** Como las dos pantallas comparten diseño, sus estilos están en `compartido/estilos/formulario.css` y cada componente los incluye con `styleUrls`.
- **Pruebas.**

- **Pruebas unitarias:** se añadieron 19, que cubren el servicio, el interceptor, el guard y las dos pantallas: validaciones, envío correcto, que la confirmación no se envía al servidor y la gestión de errores 401 y 409. En total pasan 35.
- **Prueba en el navegador:** se usó una API falsa que imita el backend. Se probó el formulario vacío, la contraseña incorrecta, el login correcto, contraseñas que no coinciden en el registro y el registro correcto.

4.8.3 Contrato con el backend

El backend todavía no tiene autenticación. El frontend espera estos dos endpoints, que habrá que implementar en Spring Boot:

Método y ruta	Cuerpo	Respuesta correcta	Errores
`POST /api/auth/login`	`{ email, password }`	`200 { token, usuario: { id, nombre, email } }`	`401` si los datos no son correctos
`POST /api/auth/registro`	`{ nombre, email, password }`	`201 { token, usuario: { id, nombre, email } }`	`409 { error }` si el email ya existe; `400` en las validaciones

Además, la configuración CORS del backend debe permitir la cabecera `Authorization`.

4.8.4 Por qué

- **Sesión guardada en el servicio, con *signals*:** cualquier pantalla puede saber al momento quién ha entrado, y la cabecera se actualiza sola al entrar o salir.
- **Validar en el navegador antes de enviar:** el usuario recibe respuesta inmediata y se evitan peticiones inútiles al servidor. El backend seguirá validando igualmente por seguridad.
- **El guard no se ha aplicado todavía a ninguna ruta.** Mientras el backend no tenga login, proteger la app impediría usarla. Cuando exista, basta con añadir `canActivate: [authGuard]` a las rutas privadas.

4.8.5 Problemas encontrados

- No se puede arrancar el backend en este equipo (no tiene Java ni MongoDB), y además todavía no tiene endpoints de autenticación. Se resolvió con pruebas que simulan el servidor (`HttpTestingController`) y, para las pruebas en el navegador, con una API falsa que sigue el contrato descrito arriba.
- Al añadir el servicio `Auth` a la pantalla principal de la app, sus pruebas necesitaban `HttpClient`. Se añadieron `provideHttpClient()` y `provideHttpClientTesting()` a esas pruebas.

4.9 PS-21 · Servicio HTTP base de Angular

Autor: Asier García · **Fecha:** 21/10/2026 · **Issue:** PS-21

4.9.1 Qué se ha hecho

Se ha creado la capa que comunica el frontend con el backend: un servicio HTTP base (`Api`) que usarán todos los demás servicios, un interceptor que traduce los errores del servidor a mensajes en español y la configuración de la dirección de la API por entornos.

4.9.2 Cómo se ha hecho

- **Entornos.** Con `ng generate environments` se crearon `src/environments/environment.t` (producción) y `environment.development.ts` (desarrollo). Ambos guardan `apiUrl`, que de momento apunta al backend Spring Boot local (`http://localhost:8080/api`). Angular sustituye un archivo por otro automáticamente según el tipo de compilación.
- **Servicio `Api`** (`core/servicios/api.ts`). Envuelve el `HttpClient` de Angular con cuatro métodos genéricos (`get`, `post`, `put` y `delete`). Recibe solo la ruta relativa (por ejemplo `'rutas'`) y construye la URL completa. Se usa el nuevo decorador `@Service()` de Angular 22, que registra el servicio automáticamente en la inyección de dependencias.
- **Interceptor de errores** (`core/interceptores/errores-interceptor.ts`). Es un interceptor funcional que se ejecuta en todas las peticiones. Cuando una falla, convierte el `HttpErrorResponse` en un objeto `ErrorApi` (`core/modelos/error-api.ts`) con el código, un mensaje en español y, si los hay, los errores de validación por campo. Está adaptado a las respuestas reales del backend de Adam: `{ error: '...' }` en los errores generales y `{ campo: 'mensaje' }` en las validaciones (400).
- **Registro.** En `app.config.ts` se añadió `provideHttpClient(withFetch(), withInterceptors([erroresInterceptor]))`.
- **Pruebas.** Con `HttpTestingController` se simulan las respuestas del servidor sin necesidad de tener el backend arrancado. Se comprueba que se construyen bien las URL y los parámetros, que cada método usa el verbo HTTP correcto y que los errores 0 (sin conexión), 400, 404 y 500 se traducen correctamente. Se añadieron 6 pruebas y todas pasan.

4.9.3 Por qué

- **Un único punto de acceso al backend:** si cambia la dirección del servidor, se cambia en un solo archivo, y ningún componente necesita conocer `HttpClient`.
- **Errores centralizados:** las pantallas solo tienen que mostrar `error.mensaje`, sin repetir en cada una la lógica para interpretar los códigos HTTP. El usuario recibe siempre mensajes comprensibles en español.
- **`withFetch()`:** hace que Angular use la API `fetch` del navegador, que es la opción recomendada actualmente.

4.9.4 Problemas encontrados

- Angular 22 genera los servicios con el decorador `@Service()` en lugar del clásico `@Injectable({ providedIn: 'root' })`. Se consultó su definición para confirmar que el servicio se registra automáticamente en toda la aplicación.
- El backend no puede arrancarse en este equipo porque no tiene Java ni MongoDB. Por eso las pruebas simulan el servidor con `HttpTestingController`.

4.10 Flujo de trabajo con ramas y Pull Requests

Autor: Asier García · **Fecha:** 26/10/2026 · **Issue:** —

4.10.1 Qué se ha hecho

Se ha definido y documentado el flujo de trabajo en GitHub que exige el enunciado del proyecto (sección 5). Cada historia se desarrolla en su propia rama, nadie sube cambios directamente a `main`, y todo se integra mediante una Pull Request que revisa y aprueba otro desarrollador. Este cambio es el primero que se ha subido siguiendo el nuevo flujo.

4.10.2 Cómo se ha hecho

- **Reglas comunes en `CLAUDE.md`.** Se añadió una sección "Flujo de Git" que el asistente de cada desarrollador lee al abrir el proyecto, de modo que los tres trabajan igual sin necesidad de recordarlo cada vez:
  - **Ramas:** una por historia, con el nombre `PS-XX-descripcion-corta` (clave de Jira y 2–4 palabras), creada siempre desde `main` actualizado.
  - **Commits:** pequeños y frecuentes, con la clave de Jira en el mensaje (`PS-XX: descripción`).
  - **Integración:** antes de subir, se integra en la rama lo último de `main` (`git fetch` + `git merge origin/main`), de forma que el historial muestra que se incorpora el trabajo de los compañeros.
  - **Pull Request:** se sube solo la rama y se abre la Pull Request hacia `main`. Nunca se hace el merge de la PR propia: la aprueba otro DT y se fusiona en GitHub con *merge commit*, para que la rama quede visible en el historial.
- **Comandos del equipo.**
  - `/empezar PS-XX`: actualiza `main`, resume el estado del proyecto, avisa de las ramas abiertas de los compañeros y crea (o retoma) la rama de la tarea.
  - `/terminar`: escribe la entrada de memoria, ejecuta el build y los tests, hace commits por pasos, integra `main`, sube la rama y prepara la Pull Request.
  - `/revisar PS-XX` (nuevo): descarga la rama de un compañero, la compila, la prueba y redacta una revisión lista para pegar en GitHub, sin modificar su trabajo.
- **Plantilla de Pull Request** (`.github/pull_request_template.md`). Incluye la clave de Jira, cómo probar el cambio y la Definición de Hecho del enunciado, incluida la accesibilidad básica.
- **README.** La guía diaria del equipo explica ahora el ciclo completo: `/empezar` → trabajo en la rama → `/terminar` → revisión de otro DT → fusión.

4.10.3 Por qué

- **Lo exige el enunciado:** “nadie sube cambios directamente a `main`”, “cada historia se desarrolla en su propia rama” y cada DT debe tener commits y Pull Requests propios en todos los sprints. Además, la Definición de Hecho exige que el código esté integrado en `main` mediante una Pull Request revisada.
- **Deja constancia del trabajo:** las ramas y las Pull Requests muestran quién ha hecho cada historia, quién la ha revisado y cuándo se ha integrado, lo que se usará en el apartado “Trabajo en GitHub” de la memoria final.
- **Revisión entre compañeros:** las revisiones permiten detectar errores antes de que lleguen a `main` y que todo el equipo conozca el código de los demás. Esto también lo pide el enunciado para el código generado con IA.

4.10.4 Problemas encontrados

- Las tareas DT2, PS-20, PS-21 y PS-26 se subieron directamente a `main` antes de conocer esta norma. No se ha reescrito ese historial, porque exigiría `git push --force` sobre la rama común. A partir de este cambio, todo entra por Pull Request.
- Para que GitHub impida de verdad subir a `main` sin revisión, el propietario del repositorio debe activar una regla de protección de la rama (*Require a pull request before merging*, con una aprobación).

5 Conclusiones

(Se completa al final del proyecto: resultado obtenido, qué se aprendió, qué se mejoraría.)